National Forensic Sciences University

School of Cybersecurity and Digital Forensics

M.Sc. DFSI - Semester I

MSDF26-PC-103 Programming for Digital Forensics

Lab Assignment #3: Python Lists, Sets and Dictionaries

Objective

1. Perform essential operations on Python lists, sets and dictionaries.
2. Use indexing, slicing, iteration and comprehensions to process data.
3. Analyse duplicate observations and correlate artifacts across devices.
4. Manage evidence records using nested collections.
5. Validate input and preserve original data during processing.

General Instructions

- Complete all eight tasks using Python 3.
- Use functions and a menu-driven interface for each task
- Use only synthetic data. No external packages are required.
- Preserve the original dataset and perform modifications on working copies.
- Handle invalid input, empty collections, missing keys and absent values.
- Display set-derived results in sorted order.
- Submit code and labelled outputs showing normal, boundary and invalid cases.
- Explain your choice of data structures during evaluation.

Sr No	Description
Task-1	**Evidence List Manipulation**

	Develop a program to maintain an ordered list of evidence filenames and their corresponding sizes.
	
	`filenames = []`
	
	`sizes_kb = []`
	
	Required Operations
	
	1. Verify that both lists contain the same number of elements before processing.
	2. Display:

	○ First and last records. ○ A record using a user-provided positive or negative index. ○ First four records and last three records. ○ Alternate records. ○ Records in reverse order using slicing. 3. Accept start, stop and step values from the user and display the corresponding slice. Reject a zero step. 4. Normalise filenames by removing surrounding whitespace and converting them to lowercase using a list comprehension. 5. Remove empty filenames and their corresponding sizes together. 6. Demonstrate: ○ append() to add one record. ○ extend() to add multiple records. ○ insert() to add a record at a specified position. ○ Index assignment to modify a record. ○ Slice assignment to replace a range of records. 7. Demonstrate removal using remove(), pop() and del. Display the removed filename and size. 8. Demonstrate clear() on a temporary copy. 9. Use membership testing, count() and index() to search for a filename. Use enumerate() to display all its positions. **Conditions** • Keep filenames and sizes synchronised after every operation. • Reject negative sizes. • Missing filenames and invalid indices must produce meaningful messages. • An empty dataset must not cause the program to terminate unexpectedly.
Task-2	**Advanced List Analysis and Nested Lists**
	Problem Statement: Extend the evidence analyser to support duplicate analysis, sorting, batching and independent copies. Use the normalised dataset from Task 1. **Required Operations** 1. Represent each observation as a nested list: `[source_index, filename, size_kb]` 2. Generate a unique filename list while preserving first-occurrence order. Do not use a set or dictionary for this operation. 3. Display every filename with its frequency. 4. Demonstrate the difference between: • sort() • sorted() • reverse() • Reverse slicing.

	5. Sort records by: • Filename ascending. • Size descending. • Size descending, filename ascending and original index ascending. 6. Use a list comprehension to select records exceeding a user-provided size threshold. 7. Calculate the number of records, total size, average size, minimum size and maximum size. 8. Concatenate two evidence batches. Demonstrate list repetition on a separate example. 9. Divide records into batches of size n. Retain the final incomplete batch and reject n <= 0. 10. Rotate the list right by k positions using slicing. Support negative values as left rotation. 11. Demonstrate assignment aliasing, copy() and an independent copy of nested records. Show how an inner modification affects each version. 12. Explain and correct the shared-row problem in: matrix = [[0] * 3] * 2
Task-3	**Set Operations and Device Inventory Comparison**

	Develop a program to compare unique filenames recovered from three devices.
	`device_A = {`
	`"report.pdf", "photo.jpg", "log.txt",`
	`"archive.zip", "script.py"`
	`}`
	`device_B = {`
	`"report.pdf", "log.txt", "notes.txt", "backup.zip"`
	`}`
	`device_C = {`
	`"photo.jpg", "report.pdf", "notes.txt", "invoice.csv"`
	`}`
	Required Operations
	1. Create a set from a list containing repeated filenames and show how duplicates are removed.
	2. Create an empty set and explain why {} creates a dictionary.
	3. Perform membership testing and iteration. Display results in sorted order.
	4. Demonstrate:
	○ add()
	○ update()

	○ remove() ○ discard() ○ pop() ○ clear() ○ copy() 5. Demonstrate the behaviour of remove() and discard() when an element is absent. 6. Handle pop() on an empty set. Do not assume which element a nonempty set returns. 7. Compute: ○ All filenames found across the devices. ○ Filenames common to all three devices. ○ Filenames in A but absent from B. ○ Filenames in B but absent from A. ○ Filenames present in A or B, but not both. 8. Demonstrate both operators and named methods for union, intersection, difference and symmetric difference. 9. Find filenames present on **exactly two devices** and identify those devices.
Task-4	**Advanced Set Correlation**
	Extend Task 3 to analyse exclusive artifacts, watchlist matches and inventory similarity. watchlist = {"script.py", "macro.docm", "archive.zip"} 1. Identify filenames found on exactly one device and display the device name. 2. Find filenames absent from A but present on B or C. 3. Identify watchlist matches for every device. 4. Demonstrate: ○ issubset() ○ issuperset() ○ isdisjoint() ○ Proper subset and proper superset comparisons using < and >. 5. Demonstrate these operations on independent copies: ○ update() ○ intersection_update() ○ difference_update() ○ symmetric_difference_update() 6. Compare each result with its non-mutating equivalent. 9. Use a set comprehension to extract distinct file extensions. Report filenames without an extension separately. 10. For this assignment, use these conventions: ○ Two empty inventories: similarity 1.0. ○ Empty versus nonempty inventory: similarity 0.0. 11. Create an immutable inventory using frozenset and use it as a dictionary key. 12. Demonstrate why a mutable set cannot be a dictionary key or an element of another set.

Task-5	**Dictionary-Based Evidence Register**

Create a nested dictionary that stores evidence records using unique evidence IDs.

```
evidence = {
    "E001": {
        "filename": "report.pdf",
        "device": "A",
        "size_kb": 120,
        "status": "pending",
        "tags": ["document"]
    },
    "E002": {
        "filename": "photo.jpg",
        "device": "A",
        "size_kb": 540,
        "status": "reviewed",
        "tags": ["image"]
    },
    "E003": {
        "filename": "report.pdf",
        "device": "B",
        "size_kb": 120,
        "status": "pending",
        "tags": ["document", "duplicate-name"]
    }
}
```

1. Create another register from separate ID and record lists using zip(). Check equal lengths and unique IDs before conversion

	2. Access records using square brackets and get(). Demonstrate missing-key handling. 3. Insert a new evidence record. Reject an existing ID unless the user selects an explicit update operation. 4. Update individual nested fields and multiple fields using update(). 5. Display old and new values whenever a record changes. 6. Iterate using keys(), values() and items(). 7. Search for: • Evidence ID. • Exact normalised filename. • Device. • Status. 8. Return every matching record when multiple records share a filename. 9. Demonstrate deletion using del and pop(). 10. Demonstrate popitem() and clear() on temporary copies. Explain that popitem() removes the last-inserted pair in current Python 3.
Task-6	**Dictionary Aggregation, Merging and Copying**
	Extend the evidence register to generate summaries and handle conflicting imports. 1. Build a filename frequency dictionary using get(). 2. Build a device-to-evidence-ID-list dictionary using setdefault(). 3. Calculate:- Record count per device. o Total size per device. o Record count per status. o Use dictionary comprehensions to create:- A dictionary containing only pending records. 4. An evidence-ID-to-size dictionary. o Display records sorted by size descending, with evidence ID ascending as the tie-breaker. o Create a second register containing:- Two new IDs. 5. One existing ID with conflicting values. 6. Merge nonconflicting records. Retain the existing version of a conflicting ID and record both versions in a conflict list. 7. Separately demonstrate overwriting behaviour using update() and dictionary unpacking. 8. Create a filename-to-device-list mapping that preserves multiple devices for the same filename. 9. Demonstrate the difference between a shallow copy and copy.deepcopy() by modifying a nested tags list. 10. Create counters using: dict.fromkeys(evidence_ids, 0)

	11. Demonstrate the shared mutable default problem with: dict.fromkeys(evidence_ids, [])

Submission

Prepare a report containing the following sections:

1. Python source code.
2. A brief description of the approach.
3. Labelled screenshots or copied execution outputs.